

cornea

Builds a 2D axisymmetric cornea section bounded by two conic surfaces, meshes it by mapping a structured square mesh onto it, and exports a tagged Gmsh mesh that carries, per element, the local orientation frame (circumferential and through-thickness directions) of the mapping.

Quick start

0. Compile README.MD

For readability, compile this readme in a pdf using pandoc :

```
pandoc README.md -o README.pdf
```

1. Create the Python environment

From the `cornea_meshing_package/` folder (the one containing `pyproject.toml`):

```
python3 -m venv cornea_msh_env           # create the environment
source cornea_msh_env/bin/activate       # activate it (Linux / macOS)
pip install --upgrade pip
pip install -e .                         # installs the package and its dependencies
```

On Windows, activate with `cornea_msh_env\Scripts\activate` instead of the `source` line.

`pip install -e .` reads the dependencies from `pyproject.toml` (numpy, scipy, matplotlib, meshio) and installs them inside `cornea_msh_env` only. The `-e` (editable) flag links the package to the source folder, so changes you make to the code apply without reinstalling.

Activate the environment again in each new terminal before using the package, and leave it with `deactivate`.

2. Run

```
python -m cornea params.json -o output
```

`params.json`:

```
{
  "R_v": 5.25,
  "R_a": 6.5,  "Q_a": -0.61,
  "R_p": 5.5,  "Q_p": -0.72,
  "h": 0.5,
  "Nx": 50, "Ny": 6
}
```

Parameter	Meaning
R_v	radial extent of the posterior face
R_a, Q_a	anterior radius of curvature and asphericity
R_p, Q_p	posterior radius of curvature and asphericity
h	central thickness
Nx, Ny	number of nodes along the radius / through the thickness

Outputs in `output/`:

File	Content
<code>cornea.msh</code>	Gmsh MSH 2.2 ASCII mesh: nodes, 4-node quads, tagged boundary lines, per-element local orientation frame (<code>circ</code> , <code>normal</code> , as 3-component vector element data)
<code>cornea_mesh.png</code>	the mapped mesh
<code>cornea_fields.png</code>	<code>circ</code> and <code>normal</code> directions, read straight back from <code>cornea.msh</code>
<code>cornea_tags.png</code>	physical groups, read back from <code>cornea.msh</code>

The console also prints checks: the limbus edge is normal to the posterior face, every element keeps a positive orientation, `circ` and `normal` are orthogonal and orientation- preserving, and the values read back from `cornea.msh` match the exact map to within text I/O precision (see §3).

Physical groups in `cornea.msh`

Name	Dimension	Tag
<code>anterior</code>	1	1
<code>posterior</code>	1	2
<code>limbus</code>	1	3
<code>central_line</code>	1	4
<code>cornea</code>	2	10

Coordinates are written as `(x, z, 0)`: `x` is the radial direction, `z` the optical axis.

Using it from Python

```

from cornea import CorneaParams, CorneaGeometry, build_nodes, element_frames, write_msh, read_msh, read_frames

p = CorneaParams.from_json("params.json")          # or CorneaParams(R_v=5.0, ...)
geom = CorneaGeometry(p)
pts, uv = build_nodes(geom, p.Nx, p.Ny)           # (Ny, Nx, 2) arrays, pts = phi(uv)
circ, normal = element_frames(geom, uv)            # (n_elem, 2) each, one pair per quad
write_msh("cornea.msh", pts, circ, normal)

mesh = read_msh("cornea.msh")                      # a plain meshio.Mesh
centroids, circ, normal = read_frames(mesh)         # read straight back, no recomputation

xz = geom.phi(0.5, 0.5)                            # exact map at any (u, v)
F = geom.F_exact(0.5, 0.5)                         # its Jacobian

```

1. Geometry: the two conics

Each face is a conic sag of radius R and asphericity Q :

$$f(x; R, Q) = \frac{x^2/R}{1 + \sqrt{1 - (1 + Q)x^2/R^2}}, \quad f'(x; R, Q) = \frac{x}{\sqrt{R^2 - (1 + Q)x^2}}$$

The faces are placed directly in their final position (apex up, opening downwards):

$$z_{ant}(x) = T - f(x; R_a, Q_a), \quad z_{post}(x) = T - h - f(x; R_p, Q_p), \quad T = h + f(R_v; R_p, Q_p)$$

so the central thickness is h and the posterior end point $P = (R_v, 0)$ sits at $z = 0$.

Limbus edge. Instead of cutting vertically at $x = R_v$, the edge follows the normal to the posterior face at P . With $s = z'_{post}(R_v)$, the unit normal pointing towards the anterior face is

$$n = \frac{(-s, 1)}{\sqrt{1 + s^2}}$$

and the anterior end point is $A = P + t^*n$, where t^* solves $z_{ant}(P_x + tn_x) = P_z + tn_z$ (Brent's method). The anterior face therefore extends slightly beyond R_v (to $x \approx 5.73$ with the default parameters).

2. Mapping: square to cornea (Coons patch)

The reference square has coordinates $(u, v) \in [0, 1]^2$:

- u : radial, $u = 0$ on the central line, $u = 1$ on the limbus;
- v : through the thickness, $v = 0$ on the posterior face, $v = 1$ on the anterior face.

The four boundaries of the cornea are parametrised on $[0, 1]$:

- $C_{post}(u)$, $C_{ant}(u)$: the two conics, parametrised by normalised arc length (cubic spline of $x(s)$), so nodes are evenly spaced along each face;
- $D_{axis}(v)$, $D_{limb}(v)$: straight segments between the corner points $P_{00} = C_{post}(0)$, $P_{01} = C_{ant}(0)$ and $P_{10} = C_{post}(1) = P$, $P_{11} = C_{ant}(1) = A$.

The map is the transfinite (Coons) interpolation of these four curves:

$$\varphi(u, v) = (1-v) C_{post}(u) + v C_{ant}(u) + (1-u) D_{axis}(v) + u D_{limb}(v) - [(1-u)(1-v)P_{00} + u(1-v)P_{10} + (1-u)v P_{01} + uv P_{11}]$$

It reproduces each boundary exactly and blends them smoothly inside. Node (i, j) of an $N_x \times N_y$ grid on the square is placed at $(x, z) = \varphi(i/(N_x - 1), j/(N_y - 1))$.

Because u points outward and v points from posterior to anterior (upward), φ preserves orientation ($\det F > 0$): quads that are counter-clockwise in (u, v) are counter-clockwise in (x, z) . The writer checks this and raises an error on any inverted element.

If your solver works with displacements instead, `geom.displacement(X, Y, L, H)` gives $\varphi(X/L, Y/H) - (X, Y)$ for nodes of a square $[0, L] \times [0, H]$; keep only the boundary nodes to use it as a Dirichlet condition.

3. The local orientation frame: circ and normal

Definition

At any (u, v) , the Jacobian of the map is

$$F = \frac{\partial(x, z)}{\partial(u, v)} = \begin{bmatrix} \partial x / \partial u & \partial x / \partial v \\ \partial z / \partial u & \partial z / \partial v \end{bmatrix}$$

and the two directions actually needed downstream are its push-forwards of the square's own axes:

Quantity defined on the square	On the cornea	Function
circumferential/radial direction $(1, 0)$	circ = $F(1, 0) / \ F(1, 0)\ $	<code>push_vector(F, [1, 0])</code>

Quantity defined on the square	On the cornea	Function
through-thickness direction $(0, 1)$	normal = $F^{-T}(0, 1)/\ F^{-T}(0, 1)\ $	<code>push_normal(F, [0, 1])</code>

- **circ**: tangent to the $v = \text{const}$ curves (including both faces) — the direction along the radius/circumference;
- **normal**: $\propto \nabla v$, exactly normal to the anterior and posterior faces, pointing from posterior to anterior.

These two are orthogonal everywhere by construction ($F^T F^{-T} = I$), which is one of the checks the run prints.

Pitfall: the mapped v -grid lines $F(0, 1)$ are *not* normal to the faces (up to about 3° off here), because the Coons map is not orthogonal — that’s why **normal** uses F^{-T} , not F .

Axisymmetric note: the circumferential direction e_θ is not affected by the 2D map; for a fibre with radial and hoop components, map only the in-plane part with **circ**.

Computing circ and normal

`element_frames(geom, uv)` evaluates the exact F (`geom.F_exact`, by central differences of φ) at each quad’s centroid (u, v) and pushes it forward directly — there is no bilinear recovery step and no dependence on any nodal field. It returns two (`n_elem`, 2) arrays, **circ** and **normal**, one pair per quad, in the same element order `write_msh` builds the quads in (*i* fast, *j* slow).

Storage: circ and normal as 3-component vector element data

`cornea.msh` carries **circ** and **normal** as **element data** (one value per quad, not per node): two vector Gmsh fields, **circ** and **normal**, each with 3 components $(v_x, v_z, 0)$ — the in-plane components as computed, the out-of-plane component fixed at 0.

Gmsh’s `$ElementData/$NodeData` post-processing blocks only support 1, 3 or 9 components per field (scalar, vector, tensor) — this is a hard rule of the file format itself, not a meshio restriction, so a literal 2-component vector isn’t a legal Gmsh data block. Padding to 3 components with a dummy 0 is the standard way to carry a 2D vector in Gmsh — exactly how the node coordinates in this same file are always written $(x, z, 0)$ even though the geometry is 2D. It also means the file is read back the same way whether **circ/normal** are genuinely 2D or 3D, with no special-casing needed downstream, and it lets Gmsh’s own viewer recognise and draw them as vectors (arrows) rather than as unrelated scalar colour maps.

This replaces the old nodal `u`, `v` fields and the Jacobian-recovery step that used to read them back (`element_jacobians()` no longer exists) — the mesh now carries the orientation frame directly, and reading it back is just slicing off the dummy third component of two vectors instead of four numbers reassembled into two.

`write_msh` and `read_msh` in `mesh_io.py` are written out explicitly rather than calling `meshio.write` / `meshio.read`, because the installed `meshio` can have two independent bugs that break that round trip on a mesh that mixes cell types the way this one always does (tagged boundary `line` elements plus interior `quad` elements):

1. **Write side (numpy ≥ 2 dependent).** `meshio`'s ASCII writer formats each value with `repr()`; with `numpy  $\geq 2$` , `repr(np.float64(0.0))` is the string `"np.float64(0.0)"` instead of `"0.0"`, corrupting the file it writes.
2. **Read side (independent of numpy).** `meshio`'s Gmsh 2.2 reader reconstructs `cell_data` by splitting one flat, file-order array back into per-cell-type blocks using `len(c)` on a `(type, array)` tuple — always 2 — instead of each block's actual element count, so any mesh with more than one cell type gets split at the wrong point. Confirmed correct in `meshio 5.0.0`, confirmed broken in 5.3.5 (the latest at time of writing): a version regression, not a one-off fluke.

Both bugs are independent of the number of components per field — switching `circ1`, `circ2`, `normal1`, `normal2` (four scalar fields) to `circ`, `normal` (two 3-component vector fields) was tested against `meshio.write/meshio.read` directly and reproduces both failures identically, so the round trip still has to be handled by hand.

`write_msh` writes plain, explicitly-formatted floats (`f"{v:.12g}"`, never `repr()`), so bug 1 never arises regardless of the installed `numpy`. `read_msh` parses `$Nodes/$Elements/$PhysicalNames/$ElementData` itself, computes the correct per-cell-block split from the actual element counts, and only then builds `meshio.Mesh(points, cells, cell_data=..., field_data=...)` — so bug 2's buggy split function is never called. The object `read_msh` returns is a completely ordinary `meshio.Mesh` (`.points`, `.cells`, `.cell_data`, `.cell_data_dict`, `.get_cells_type`, `.field_data`, ...), so `plot_mesh`, `plot_fields` and `plot_tags` need no adaptation.

Assembly for later use

`read_frames(mesh)` in `jacobian.py` is the read-side counterpart of `element_frames`:

```
centroids, circ, normal = read_frames(mesh)
```

It takes the element centroids from `mesh.points/mesh.get_cells_type("quad")`, and slices the in-plane part straight off the two vector fields:

```
circ = mesh.cell_data_dict["circ"]["quad"][:, :2],    normal = mesh.cell_data_dict["normal"]["quad"][:, :2]
```

(dropping the dummy out-of-plane third component). No Jacobian, no bilinear shape functions, no matrix inversion — the mesh already carries the final vectors. `diagnostics(circ, normal, F_ref=None)` then checks that `circ` and `normal` are unit and orthogonal and that orientation is preserved (`circ x normal > 0` everywhere); passing `geom.F_exact` at the same points as `F_ref` additionally reports the angular error against the exact map, which at this point only measures the mesh's text I/O round-trip precision (`circ/normal` were written from that same exact map in the first place), typically a few $\times 10^{-5}$ degrees with `.12g` formatting.

Package layout

```
cornea/
  params.py      CorneaParams: inputs, JSON read/write
  geometry.py    conics, limbus edge, Coons map phi, exact F
  mesh_io.py     structured nodes, element_frames (circ/normal from the exact map),
                  write_msh / read_msh (tags + circ/normal as 3-component vector element data)
  jacobian.py    push_vector / push_normal, read_frames (assembles circ/normal), diagnostics
  plots.py       plot_mesh, plot_fields, plot_tags
  __main__.py    command line: python -m cornea params.json -o output
params.json
pyproject.toml
```